Step1

1.1

1. 156
2. -100
3. 零的表示唯一 原码：0000 0000 表示 +0，1000 0000 表示 -0，有两个零。 反码：0000 0000 表示 +0，1111 1111 表示 -0，也有两个零。 补码：0000 0000 唯一表示 0，没有 -0，逻辑更干净。

   便于减法 补码中，a - b 等价于 a + (-b)，而 -b 的补码就是 ~b + 1， 这样，CPU 只需要一个加法器就能同时完成加法和减法，不需要为减法单独设计电路。

   符号位无需特殊处理 在原码和反码中，做加减法时要先判断符号、比较绝对值大小、决定结果符号，逻辑复杂。 补码中，符号位和数值位一起参与运算，硬件实现简单得多。

   表示范围多一个负数 8位补码范围：-128 ~ 127 8位原码/反码范围：-127 ~ 127（因为有两个零） 补码能多表示一个最小负数 -128，利用率更高。

   溢出判断简单 补码运算的溢出可以通过最高位进位和次高位进位的异或来判断，硬件实现非常方便。

1.2

[警告!] 温度过高！触发消防喷淋系统！

如果一个操作数是有符号的，另一个是无符号的，那么有符号操作数会被隐式转换为无符号操作数。 -10的补码被转换成一个极大的正数，因此会判断为过热。

Step2

2.1

1. 10011001
2. -103
3.

int is_safe(int x, int y) { int sum = x + y; int x_sign = (x >> 31) & 1; int y_sign = (y >> 31) & 1; int sum_sign = (sum >> 31) & 1; int result;

```
// TO DO
// 同号相加才可能溢出；若结果符号与操作数符号不同，则溢出
if (x_sign == y_sign && sum_sign != x_sign) {
    result = 0;
} else {
    result = 1;
}
```

```
    // END OF TO DO

    return result;
```

}

2.2

int final_milk(int x) { int y;

```
    // TO DO
    y = (x << 5) - (x << 1) - x;    // 32x - 2x - x = 29x
    // END OF TO DO

    return y;
```

}

Step3

3.1

1.

类型 总位数 符号位（S） 阶码位（E） 尾数位（M） float 32 位 1 位 8 位 23 位 double 64 位 1 位 11 位 52 位

2. 偏置值由阶码位数决定，公式为 Bias = 2^(k-1) - 1，其中k为阶码的位数。采用偏置表示法的核心目的是用无符号整数统一表示有符号的阶码，从而避免在浮点数的比较和运算中为阶码单独处理符号，简化硬件设计。

3. 当阶码位全为0且尾数位非0时，表示的就是非规格化数。规格化数的尾数隐含前导 1（形式为 1.f），而非规格化数的隐含位为 0（形式为 0.f）。非规格化数的实际阶码固定为 1 - Bias（float 为 -126，double 为 -1022），而非规格化数的最小规格化阶码 -126（float）。

4. 阶码全1且尾数全0。符号位为0表示+∞，为1表示-∞。它用于表示溢出或除以零等运算结果。 阶码全1且尾数非0。它表示一个无效的或未定义的运算结果，例如 0.0/0.0、负数的平方根等。

3.2

二进制位模式 编码类型 实际阶码 e (十进制) 尾数真值 M (二进制小数) 十进制最终真值 0 0110 100 规格化 6 − 7 = −1 1.100_2 = 1.5 1.5×2^{-1} = 0.75 0 0000 011 (A) 非规格化 (B) −6 (C) 0.011_2 (D) 0.375×2^{-6} = 0.005859375 0 1111 000 (E) 特殊值（正无穷） 不适用 不适用 +∞ 0 1001 010 规格化 (F) 9 − 7 = 2 (G) 1.010_2 = 1.25 (H) 1.25×2^2 = 5.0

3.3

1.

位模式：0 0000 001 十进制的值： $0.125 \times 2^{-6} = 1/8 \times 1/64 = 1/512 = 0.001953125$

位模式：0 0000 111 十进制的值： $0.875 \times 2^{-6} = 7/8 \times 1/64 = 7/512 = 0.013671875$

2.

位模式：0 1110 111 十进制的值: $1.875 \times 2^{7} = 1.875 \times 128 = 240$

位模式：0 0001 000 十进制的值: $1.0 \times 2^{-6} = 1/64 = 0.015625$

3. 最大正非规格化数 = 7/512,最小正规格化数 = 1/64 = 8/512,两者之间恰好相差一个最小单位 1/512，实现了从非规格化数到规格化数的无缝衔接。 如果非规格化数的阶码采用简单的 −Bias = −7，那么最大非规格化数 = 7/1024,最小规格化数 = 16/1024,两者之间会出现巨大的空隙（从 7/1024 直接跳到 16/1024），导致 0 附近的数值分布不连续。

Step4

4.1

#include <stdio.h>

//解析8位二进制字符串，提取阶码E和尾数M void parse_float(const char *s, int *E, int *M) { *E = 0; *M = 0; // s[0] 是符号位，忽略（题目只考虑非负规格化数） for (int i = 1; i <= 4; i++) { *E = (*E << 1) | (s[i] - '0'); } for (int i = 5; i <= 7; i++) { *M = (*M << 1) | (s[i] - '0'); } }

// 将阶码和尾数组合成8位二进制字符串 void format_float(char *out, int E, int M) { out[0] = '0'; // 符号位固定为 0 // 阶码4位 for (int i = 3; i >= 0; i--) { out[1 + (3 - i)] = ((E >> i) & 1) ? '1' : '0'; } // 尾数3位 for (int i = 2; i >= 0; i--) { out[5 + (2 - i)] = ((M >> i) & 1) ? '1' : '0'; } out[8] = '\0'; }

int main() { char s1[9], s2[9]; if (scanf("%s %s", s1, s2) != 2) { return 1; }

```
int E1, M1, E2, M2;
parse_float(s1, &E1, &M1);
parse_float(s2, &E2, &M2);

int e1 = E1 - 7;
int e2 = E2 - 7;
int f1 = 8 + M1;  // 1.MMM 对应的整数，范围 8~15
int f2 = 8 + M2;

// 对阶：取较大的阶码
int e_max = (e1 > e2) ? e1 : e2;
// 将较小阶码的尾数右移，对齐到 e_max（向零舍入，即直接截断）
if (e1 < e_max) {
    f1 >>= (e_max - e1);
}
if (e2 < e_max) {
    f2 >>= (e_max - e2);
}

int sum = f1 + f2;
```

```
    int e = e_max;

    // 规格化：尾数 >= 2 时右移，阶码加1
    while (sum >= 16) {
        sum >>= 1;
        e++;
    }
    // 如果 sum < 8，需要左移（理论上不会发生，但保留）
    while (sum < 8 && e > -7) {
        sum <<= 1;
        e--;
    }

    // 提取最终的 E 和 M
    int E_out = e + 7;
    int M_out = sum - 8;  // sum 范围 8~15，所以 M_out 范围 0~7

    // 格式化输出
    char out[9];
    format_float(out, E_out, M_out);
    printf("%s\n", out);

    return 0;
```

}

4.2

32位float尾数只有2 位，加上隐含的1，一共 24 位有效二进制位，大约对应 7 位十进制有效数字。也就是说，float 只能精确表示约 7 位有效数字。超过这个范围的数字，只能近似存储。

1e20f是10^20，它有21位十进制数字，远超float的7位精度。 15.0f 只有 2 位有效数字。两者相差约 19 个数量级。

两个浮点数相加时，不能直接把尾数相加，必须先对齐阶码（对阶）。先将阶码较小的数的尾数右移，直到它的阶码与较大数相同。右移时，移出的位直接丢弃（向零舍入）。

要对阶，把1.5×10^1 变成 0.000...00015 × 10^20，其中小数点后需要补 19 个 0。但 float 的尾数只能存 7 位左右，后面补的 19 个 0 根本存不下，15 就被完全移出，变成 0。所以计算就变成了1e20 + 15 = 1e20，这就是浮点数的“大数吸收”现象。

#include <stdio.h>

int main() { // 以“分”为单位，1 元 = 100 分，把小数转换为整数 // 这里用 1e10 元 = 1e12 分，确保在 long long 范围内（long long 最大约 9.22e18） long long boss_wallet_cents = 1000000000000LL; long long daily_expense_cents = -1000000000000LL; long long milk_tea_price_cents = 1500LL;

```
    // 算法 1：先算净值，再加奶茶钱
    long long revenue1 = (boss_wallet_cents + daily_expense_cents) +
    milk_tea_price_cents;
```

```
    // 算法 2：先算支出加奶茶钱，再加初始资产
    long long revenue2 = boss_wallet_cents + (daily_expense_cents +
    milk_tea_price_cents);

    printf("算法1: %lld 分 = %.2f 元\n", revenue1, revenue1 / 100.0);
    printf("算法2: %lld 分 = %.2f 元\n", revenue2, revenue2 / 100.0);

    return 0;
```

}